Summarize

Chapter 13. Coordinated Batch Processing

Chapter 12 described a number of patterns for splitting and chaining queues together to achieve more complex batch processing. Duplicating and producing multiple different outputs is often an important part of batch processing, but sometimes it is equally important to pull multiple outputs back together in order to generate some sort of aggregate output. A generic illustration of such a pattern is shown in Figure 13-1.

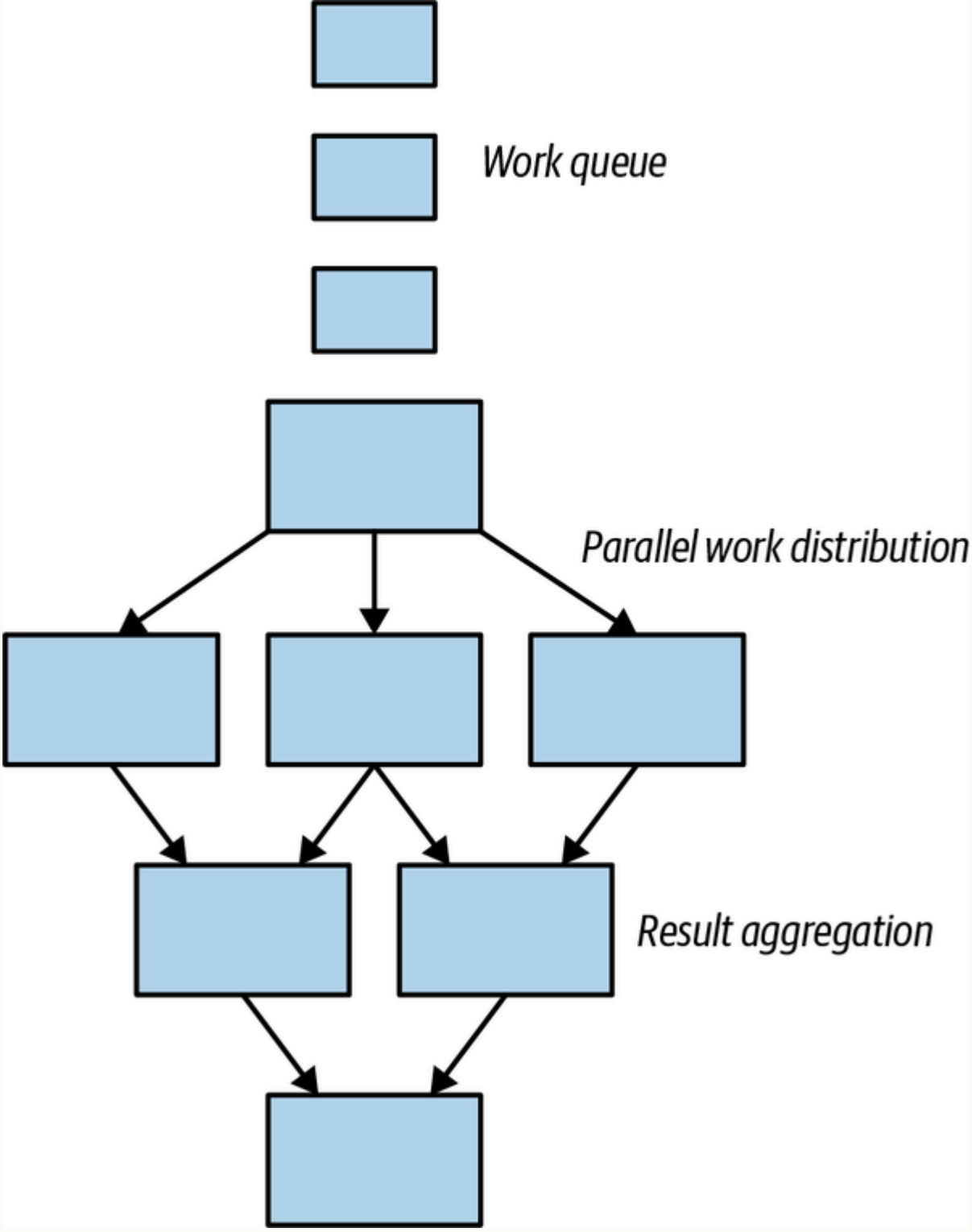


Figure 13-1. A generic parallel work distribution and result aggregation batch system

Probably the most canonical example of this aggregation is the *reduce* part of the MapReduce pattern. It's easy to see that the map step is an example of sharding a work queue, and the reduce step is an example of coordinated processing that eventually reduces a large number of outputs down to a single aggregate response. However, there are a number of different aggregate patterns for batch processing, and this chapter discusses a number of them in addition to real-world applications.

Join (or Barrier Synchronization)

In previous chapters, we saw patterns for breaking up work and distributing it in parallel on multiple nodes. In particular, we saw how a sharded work queue could distribute work in parallel to a number of different work queue shards. However, sometimes when processing a workflow, it is necessary to have the complete set of work available to you before you move on to the next stage of the workflow.

One option for doing this was shown in Chapter 12, which was to merge multiple queues together. However, merge simply blends the output of two work queues into a single work queue for additional processing. While the merge pattern is sufficient in some cases, it does not ensure that a complete data set is present prior to the beginning of processing. This means that there can be no guarantees about the completeness of the processing being performed, as well as no opportunity to compute aggregate statistics for all of the elements that have been processed.

To give a concrete example of the necessity of the *join* pattern, consider the creation of a digitally animated film. Each frame of the film is an independent work item; it can be rendered in parallel without reference to any of the other frames in the movie. However, consider transforming each frame of the movie into one second of video. You clearly need to have all thirty frames that make up that one second present before they are transformed into video.

We need a stronger, coordinated primitive for batch data processing, and that primitive is the join pattern. Join is similar to joining a thread. The basic idea is that all of the work is happening in parallel, but work items aren't released out of the join until all of the work items that are processed in parallel are completed. This is also generally known as *barrier* synchronization in concurrent programming. An illustration of the join pattern for a coordinated batch is shown in Figure 13-2.

Coordination through join ensures that no data is missing before some sort of aggregation phase is performed (e.g., finding the sum of some value in a set). The value of the join is that it ensures that all of the data in the set is present. The downside of the join pattern is that it requires that all data be processed by a previous stage before subsequent computation can begin. This reduces the parallelism that is possible in the batch workflow, and thus increases the overall latency of running the workflow, as discussed in Chapter 12. This latency increase can be compensated for by trading compute for time and duplicating the processing of all of the work items to reduce the impact of tail latency.

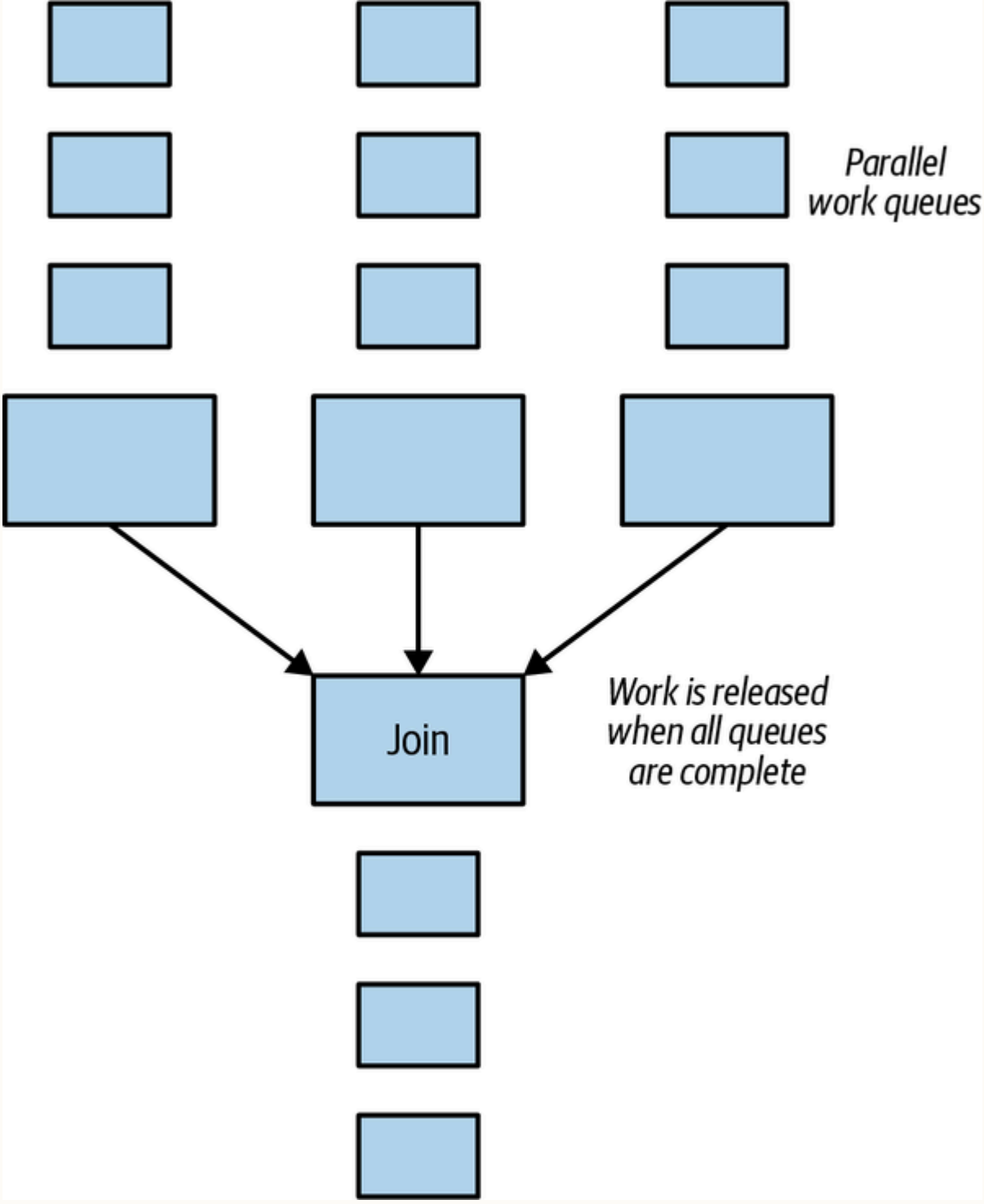


Figure 13-2. The join pattern for batch processing

Reduce

If sharding a work queue is an example of the *map* phase of the canonical MapReduce algorithm, then what remains is the *reduce* phase. Reduce is an example of a coordinated batch processing pattern because it can happen regardless of how the input is split up, and it is used similar to join; that is, to group together the parallel output of a number of different batch operations on different pieces of data.

However, in contrast to the join pattern described previously, the goal of reduce is not to wait until all data has been processed, but rather to optimistically merge together all of the parallel data items into a single comprehensive representation of the full set.

With the reduce pattern, each step in the reduce merges several different outputs into a single output. This stage is called “reduce” because it reduces the total number of outputs. Additionally, it reduces the data from a complete data item to simply the representative data necessary for producing the answer to a specific batch computation. Because the reduce phase operates on a range of input and produces a similar output, the reduce phase can be repeated as many or as few times as necessary in order to successfully reduce the output down to a single output for the entire data set.

This is a fortunate contrast to the join pattern, because unlike join, it means that reduce can be started in parallel while there is still processing going on as part of the map/shard phase. Of course, in order to produce a complete output, all of the data must be processed eventually, but the ability to begin early means that the batch computation executes more quickly overall.

Hands On: Count

To understand how the reduce pattern works, consider the task of counting the number of instances of a particular word in a book. We can first use sharding to divide up the work of counting words into a number of different work queues. As an example, we could create 10 different sharded work queues with 10 different people responsible for counting words in each queue. We can shard the book among these 10 work queues by looking at the page number. All pages that end in the number 1 will go to the first queue, all pages that end in the number 2 will go to the second, and so forth.

Once all of the people have finished processing their pages, they write down their results on a piece of paper. For example, they might write:

```
a: 50
the: 17
cat: 2
airplane: 1
...
```

This can be output to the reduce phase. Remember that the reduce pattern reduces by combining two or more outputs into a single output.

Given a second output:

```
a: 30
the: 25
```

```
dog: 4
airplane: 2
...
```

The reduction proceeds by summing up all of the counts for the various words, in this example producing:

```
a: 80
the 42
dog: 4
cat: 2
airplane: 3
...
```

It's clear to see that this reduction phase can be repeated on the output of previous reduce phases until there is only a single reduced output left. This is valuable since this means that reductions can be performed in parallel.

Ultimately, in this example you can see that the output of the reduction will be a single output with the count of all of the various words that are present in the book. Of course, the number of times reduce is run has an impact on the overall performance of the batch processing. Too many reduce operations cause significant wasted computation, while too few begin to resemble the synchronization of the join pattern.

Sum

A similar but slightly different form of reduction is the summation of a collection of different values. This is like counting, but rather than simply counting one for every value, you actually add together a value that is present in the original output data.

Suppose, for example, you want to sum the total population of the United States. Assume that you will do this by measuring the population in every town and then summing them all together.

A first step might be to shard the work into work queues of towns, sharded by state. This is a great first sharding, but it's clear that even when distributed in parallel, it would take a single person a long time to count the number of people in every town. Consequently, we perform a second sharding to another set of work queues, this time by county.

At this point, we have parallelized first to the level of states, then to the level of counties, and then each work queue in each county produces a stream of outputs of (town, population) tuples.

Now that we are producing output, the reduce pattern can kick in.

In this case, the reduce doesn't even really need to be aware of the two-level sharding that we performed. It is sufficient for the reduce to simply grab two or more output items, such as `(Seattle, 4,000,000)` and `(Northampton, 25,000)`, and sum them together to produce a new output `(Seattle-Northampton, 4,025,000)`. It's clear to see that, like counting, this reduction can be performed an arbitrary number of times with the same code running at each interval, and at the end, there will only be a single output containing the complete population of the United States. Importantly, again, nearly all of the computation required is happening in parallel.

Histogram

As a final example of the reduce pattern, consider that while we are counting the population of the United States via parallel sharding/mapping and reducing, we also want to build a model of the average American family. To do this, we want to develop a *histogram* of family size; that is, a model that estimates the total number of families with zero to 10 children. We will perform our multilevel sharding exactly as before (indeed, we can likely use the same workers).

However, this time, the output of the data collection phase is a histogram per town:

```
0: 15%
1: 25%
2: 45%
3: 10%
4: 5%
```

From the previous examples, we can see that if we apply the reduce pattern, we should be able to combine all of these histograms to develop a comprehensive picture of the United States. At first blush, it may seem quite difficult to understand how to merge these histograms, but when combined with the population data from the summation example, we can see that if we multiply each histogram by its relative population, then we can obtain the total population for each item being merged. If we then divide this new total by the sum of the merged populations, it is clear that we can merge and update

multiple different histograms into a single output. Given this, we can apply the reduce pattern as many times as necessary until a single output is produced.

Hands On: An Image Tagging and Processing Pipeline

To see how coordinated batch processing can be used to accomplish a larger batch task, consider the job of tagging and processing a set of images. Let us assume that we have a large collection of images of highways at rush hour, and we want to count both the numbers of cars, trucks, and motorcycles, as well as distribution of the colors of each of the vehicles. Let us also suppose that there is a preliminary step to blur the license plates of all of the vehicles to preserve anonymity.

The images are delivered to us as a series of HTTPS URLs where each URL points to a raw image. The first stage in the pipeline is to find and blur the license plates. To simplify each task in the work queue, we will have one worker that detects a license plate, and a second worker that blurs that location in the image. We will combine these two different worker containers into a single container group using the multiworker pattern described in Chapter 12.

This separation of concerns may seem unnecessary, but it facilitates container image reuse. Imagine if the API to our container is an image and a box to blur. The same container that blurs license plates can be reused to blur people's faces in a different pipeline.

Additionally, to ensure reliability and to maximize parallel processing, we will shard the images across multiple worker queues. This complete workflow for sharded image blurring is shown in Figure 13-3.

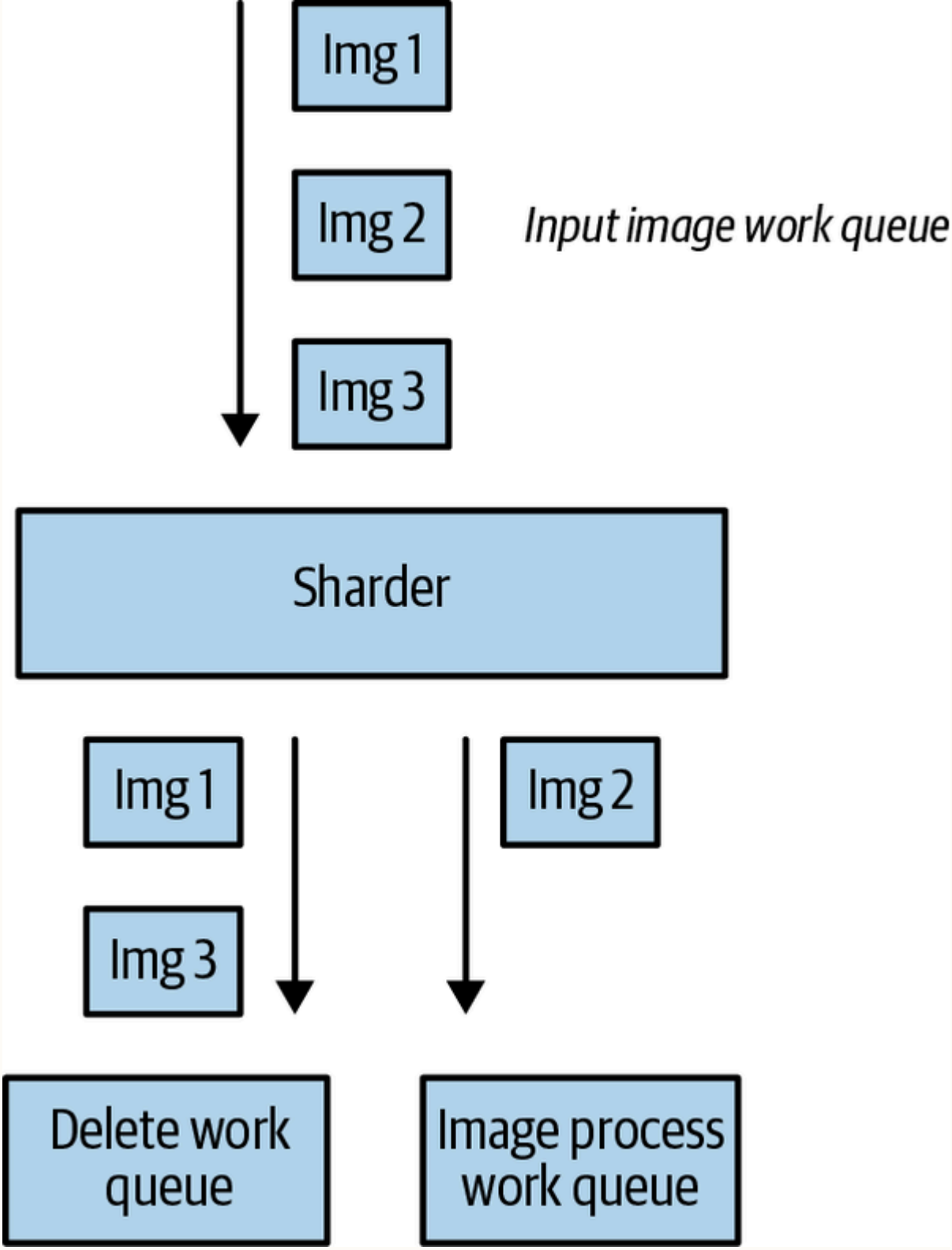


Figure 13-3. The sharded work queue and the multiple blurring shards

Once each image has been successfully blurred, we will upload it to a different location, and we will then delete the originals. However, we don't want to delete the original until *all* of the images have been successfully blurred in case there is some sort of catastrophic failure and we need to rerun this entire pipeline. Thus, to wait for all of the blurring to complete, we use the join pattern to merge the output of all of the sharded blurring work queues into a single queue that will only release its items after all of the shards have completed the work.

Now we are ready to delete the original images as well as begin work on vehicle model and color detection. Again, we want to maximize the throughput of this pipeline, so we will use the copier pattern from Chapter 12 to duplicate the work queue items to two different queues:

- A work queue that deletes the original images
- A work queue that identifies the type of vehicle (car, truck, motorcycle) and the color of the vehicle

Figure 13-4 shows these stages of the processing pipeline.

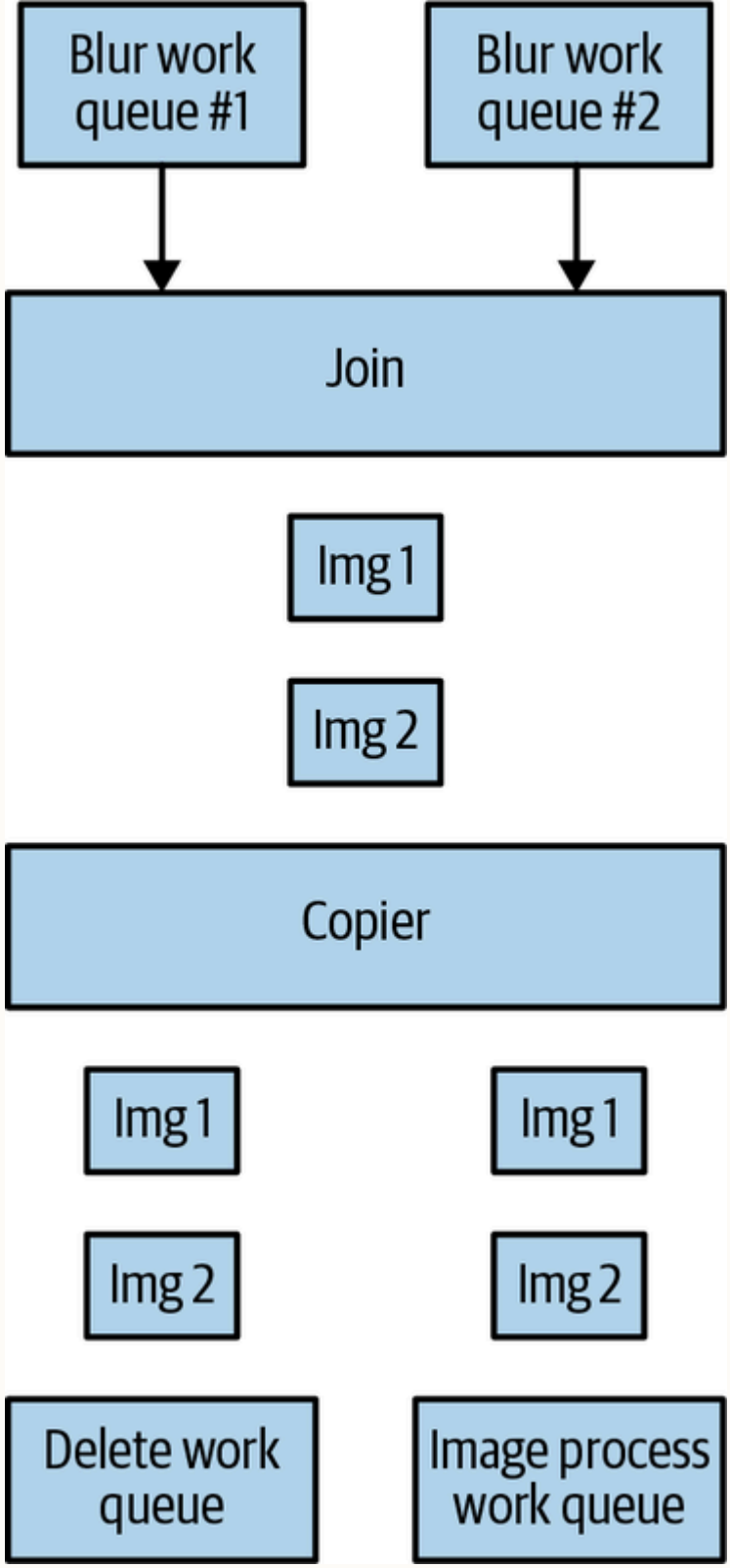


Figure 13-4. The output join, copier, deletion, and image recognition parts of the pipeline

Finally, we need to design the queue that identifies vehicles and colors and aggregates these statistics into a final count. To do this, we first again apply the shard pattern to distribute the work out to a number of queues. Each of these queues has two different workers: one that identifies the location and type of each vehicle and one that identifies the color of a region.

We will again join these together using the multiworker pattern described in Chapter 12. As before, the separation of code into different containers enables us to reuse the color detection container for multiple tasks beyond identifying the color of the vehicles.

The output of this work queue is a JSON tuple that looks like this:

```
{
  "vehicles": {
    "car": 12,
    "truck": 7,
    "motorcycle": 4
  },
  "colors": {
    "white": 8,
    "black": 3,
    "blue": 6,
    "red": 6
  }
}
```

This data represents the information found in a single image. To aggregate all of this data together, we will use the reduce pattern described previously and made famous by MapReduce to sum everything together just as we did in the count example above. At the end, this reduce pipeline stage produces the final count of images and colors found in the complete set of images.

Summary

This chapter describes *coordinated batch processing*, which is used to synchronize workflows for preconditions or to combine output from previous workflows into a merged output. The *join* pattern is used as a barrier synchronizer to ensure that all of the necessary output is ready before proceeding to the next stage. Reduce combines output from multiple shards into an output that can be reduced further until the final output is ready. Sum and histogram are specific types of reduce operations that are useful for obtaining specific statistical results from your data. Taken together, these patterns for coordination in batch processing can ensure that you can build the workflow that is necessary for any batch operation or calculation you need to perform.

Previous chapter

< 12. Event-Driven Batch Processing

Next chapter

V. Universal Concepts >